Министерство Образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Инженерно-экономический факультет
Кафедра экономической информатики

ОТЧЕТ
к лабораторной работе №1
на тему

ИСПОЛЬЗОВАНИЕ ПОТОКОВ ВВОДА/ВЫВОДА ДАННЫХ

по курсу
«Объектно-ориентированное проектирование
и программирование»

Выполнил: А. А. Левковский
студент группы 572301

Проверила: М. Н. Салапура

Минск 2026

Цель: изучить понятие потока, организацию ввода данных из потока и вывода в поток, контроль состояния потока и исправление ошибок, неформатированный ввод-вывод, манипуляторы потоков (стандартные и определяемые пользователем).

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Поток в C++ – это объект потокового класса, через который выполняется ввод или вывод данных; разные классы отвечают за разные типы потоков, например ifstream связан с файловым вводом. Использование потоковых классов вместо функций C (printf, scanf и т.п.) удобнее: каждый тип сам определяет, как его выводить, а операторы << и >> можно перегружать для своих классов, что упрощает работу и уменьшает число ошибок. Иерархия потоковых классов довольно сложна: базовым является ios_base, от него происходят basic_ios, затем basic_istream, basic_ostream и basic_iostream, а от них – файловые (ifstream, ofstream, fstream) и строковые потоки. Оператор извлечения >> объявлен в istream, оператор вставки << – в ostream; cout – стандартный выходной поток (экран), cin — стандартный входной поток (клавиатура), классы консольного ввода-вывода находятся в iostream, файловые – в fstream, манипуляторы – в iomanip. Класс ios_base хранит константы и флаги, общие для всех потоков, например showpoint и fixed; basic_istream содержит get, getline, read, basic_ostream – put, write, а basic_iostream наследует оба и подходит для чтения и записи.

Для управления форматом применяются флаги и манипуляторы: флаги устанавливаются методом setf() и снимаются unsetf(), действуют на все последующие операции до отмены, а манипуляторы включаются прямо в цепочку вывода и действуют только на последующие данные. Флаг можно задать как cout.setf(ios::flag), снять как cout.unsetf(ios::flag), а несколько флагов объединять через |. К основным флагам относятся: boolalpha и noboolalpha (вывод bool как true/false или 1/0), dec, oct, hex (системы счисления), showbase и noshowbase (показ основания), uppercase и nouppercase (регистр шестнадцатеричных цифр и показателя степени), showpos и noshowpos (явный знак +), scientific и fixed (научная или десятичная форма), internal (знак слева, число справа), showpoint и noshowpoint (показ дробной части), skipws и noskipws (пропуск или чтение пробелов), left и right (выравнивание по левому или правому краю). Манипуляторы бывают без аргументов (dec, hex, endl, fixed, left, right, showpoint и др.) и с аргументами (setw, setprecision, setfill, setiosflags, resetiosflags). Из iomanip доступны resetiosflags, setiosflags, setfill, setprecision, setw; из ios_base/basic_ios – fill(), precision(), width(), setf(), unsetf().

Двухаргументная версия setf(mask, unset) сбрасывает группу флагов и устанавливает нужный: например, cout.setf(ios::left, ios::adjustfield) сбрасывает все флаги выравнивания и включает left. Для групп определены basefield (dec, oct, hex), adjustfield (left, right, internal) и floatfield (scientific, fixed). Манипуляторы и функции позволяют настраивать точность, ширину поля,

символ заполнения и выравнивание, причём манипулятор действует только на следующий за ним элемент, а функция-флаг – на все последующие.

Класс basic_istream отвечает за извлечение данных: operator>> читает форматированные данные; get(Ch) читает один символ; get(str, count) и get(str, count, delim) читают в массив до перевода строки или до указанного разделителя; getline(str, count) и getline(str, count, delim) читают строку целиком до разделителя; putback(Ch) возвращает символ обратно в поток; ignore(count, delim) пропускает до count символов или до разделителя включительно; peek() возвращает следующий символ, не извлекая его; gcount() сообщает число символов, прочитанных при последнем неформатированном вводе; read(str, count) читает заданное число символов; seekg()/tellg() управляют позицией чтения. Класс basic_ostream отвечает за вставку: operator<< выводит форматированные данные; put(Ch) помещает один символ; flush() очищает буфер; write(str, count) пишет массив символов; seekp()/tellp() работают с позицией записи. Помимо cin и cout, есть cerr – небуферизованный поток ошибок (выводится сразу, не перенаправляется) и clog – буферизованный поток ошибок.

При ошибках ввода-вывода в потоке выставляются флаги состояния: badbit – потеря целостности буфера, eofbit – достигнут конец файла, failbit – не удалось извлечь допустимое поле, goodbit – ошибок нет. Проверить состояние можно методами eof(), fail(), good(), bad(), а сбросить флаги – методом clear() (без аргументов снимает все ошибки, с аргументом устанавливает указанный флаг). Типичная схема обработки: если cin.good() – ввод корректен, иначе cin.clear() сбрасывает ошибки, а cin.ignore(10, '\n') удаляет из потока лишние символы, после чего можно запросить ввод заново. Флаг failbit возникает, например, если вместо числа введены буквы или нажат Enter; при работе со строками он появляется при установленном noskipws, когда извлечение не может получить ни одного символа. Для корректной обработки ошибок важно вовремя очищать поток, иначе все последующие операции ввода будут неудачными.

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

18. Разработать набор классов (минимум 5 классов, связи между классами: агрегация, композиция, наследование) по предметной области «Магазин одежды». Функционал программы должен позволить собрать данные о заказе.

Листинг кода:

```
//l1_class_header.h
#pragma once
#include <iostream>
#include <string>
#include <fstream>

using namespace std;
```

```
class Clothing;

class Customer {
private:
    string name;
    string phone_number;
    string address;
    int card_id;

public:
    Customer();
    Customer(string name, string phone_number, string address, int card_id);
    void print_info(ostream& os);
    ~Customer();
};

class Order {
private:
    Customer customer;
    Clothing** items;
    string order_date;
    int order_id;
    double total_price;
    int item_count;
    int capacity;

public:
    Order();
    Order(Customer customer, string order_date, int order_id);
    ~Order();

    void add_item(Clothing* item);
    void remove_item(int index);
    void edit_item(int index, Clothing* new_item);
    double calculate_total();
    void print_receipt(ostream& os);
    bool save_to_file(const string& filename);
    void clear();

    int get_item_count() const { return item_count; }
    Clothing* get_item(int index) const { return items[index]; }
};

class Clothing {
protected:
    string name;
    string brand;
    string size;
    string color;
    double price;
    int quantity;

public:
    Clothing();
    Clothing(string name, string brand, string size, string color, double price, int
quantity);

    virtual void print_info();
    virtual string get_category() = 0;
    virtual ~Clothing();

    string get_name() const { return name; }
    string get_brand() const { return brand; }
    string get_size() const { return size; }
```

```
    string get_color() const { return color; }
    double get_price() const { return price; }
    int get_quantity() const { return quantity; }

    void set_name(const string& set) { name = set; }
    void set_brand(const string& set) { brand = set; }
    void set_size(const string& set) { size = set; }
    void set_color(const string& set) { color = set; }
    void set_price(double set) { price = set; }
    void set_quantity(int set) { quantity = set; }
};

class Jacket : public Clothing {
private:
    string season;

public:
    Jacket();
    Jacket(string name, string brand, string size, string color,
        double price, int quantity, string season);

    void print_info() override;
    string get_category() override;
    ~Jacket();

    string get_season() const { return season; }
    void set_season(const string& v) { season = v; }
};

class Trousers : public Clothing {
private:
    string fit;

public:
    Trousers();
    Trousers(string name, string brand, string size, string color,
        double price, int quantity, string fit);

    void print_info() override;
    string get_category() override;
    ~Trousers();

    string get_fit() const { return fit; }
    void set_fit(const string& v) { fit = v; }
};

class Shirt : public Clothing {
private:
    string sleeve_type;

public:
    Shirt();
    Shirt(string name, string brand, string size, string color,
        double price, int quantity, string sleeve_type);

    void print_info() override;
    string get_category() override;
    ~Shirt();

    string get_sleeve_type() const { return sleeve_type; }
    void set_sleeve_type(const string& v) { sleeve_type = v; }
};

class Shop {
private:
```

```
    string shop_name;
    Clothing** catalog;
    Order** orders;
    int catalog_count;
    int catalog_capacity;
    int orders_count;
    int orders_capacity;

public:
    Shop();
    Shop(string shop_name);
    ~Shop();

    void add_to_catalog(Clothing* item);
    void remove_from_catalog(int index);
    void edit_catalog_item(int index, Clothing* newItem);

    void add_order(const Customer& customer);
    void find_item(const string& name);
    void show_catalog(ostream& os);
    void show_orders(ostream& os);
    bool save_all_to_file(const string& filename);

    void set_shop_name(const string& name) { shop_name = name; }

    int get_catalog_count() const { return catalog_count; }
    Clothing* get_catalog_item(int index) const { return catalog[index]; }

    int get_orders_count() const { return orders_count; }
    Order& get_order(int index) const { return *orders[index]; }
};

//lab1_classes.cpp
#include <iostream>
#include <fstream>
#include <iomanip>
#include <string>
#include <windows.h>
#include "l1_class_header.h"

using namespace std;

Customer::Customer() : name(""), phone_number(""), address(""), card_id(0) {}

Customer::Customer(string name, string phone_number, string address, int card_id)
    : name(name), phone_number(phone_number), address(address), card_id(card_id) {
}

void Customer::print_info(ostream& os) {
    os << "Имя: " << name
        << ", Телефон: " << phone_number
        << ", Адрес: " << address
        << ", Карта: " << card_id;
}

Customer::~Customer() {}

Order::Order()
    : customer(), items(nullptr), order_date(""), order_id(0),
    total_price(0), item_count(0), capacity(0) {
}

Order::Order(Customer customer, string order_date, int order_id)
    : customer(customer), items(nullptr), order_date(order_date),
    order_id(order_id), total_price(0), item_count(0), capacity(0) {
```

```
}

Order::~Order() {
    delete[] items;
}

void Order::add_item(Clothing* item) {
    if (item_count >= capacity) {
        int new_cap = (capacity == 0) ? 4 : capacity * 2;
        Clothing** new_items = new Clothing * [new_cap];
        for (int i = 0; i < item_count; ++i) new_items[i] = items[i];
        delete[] items;
        items = new_items;
        capacity = new_cap;
    }
    items[item_count++] = item;
}

void Order::remove_item(int index) {
    if (index < 0 || index >= item_count) return;
    for (int i = index; i < item_count - 1; ++i) items[i] = items[i + 1];
    --item_count;
}

void Order::edit_item(int index, Clothing* new_item) {
    if (index < 0 || index >= item_count || !new_item) return;
    items[index] = new_item;
}

double Order::calculate_total() {
    total_price = 0;
    for (int i = 0; i < item_count; ++i)
        if (items[i]) total_price += items[i]->get_price();
    return total_price;
}

void Order::print_receipt(ostream& os) {
    os << "Заказ №" << order_id << " от " << order_date << "\n";
    customer.print_info(os);
    os << "\nТовары:\n";

    os.setf(ios::fixed, ios::floatfield);
    os << setprecision(2);
    os.setf(ios::showpoint);
    os.setf(ios::right, ios::adjustfield);

    os << setw(71) << setfill('-') << "" << setfill(' ') << endl
        << setw(7) << "№" << " | "
        << setw(22) << "Название" << " | "
        << setw(17) << "Категория" << " | "
        << setw(14) << "Цена" << " | " << endl
        << setw(71) << setfill('-') << "" << setfill(' ') << endl;

    for (int i = 0; i < item_count; ++i) {
        os << setw(7) << i + 1 << " | "
            << setw(22) << items[i]->get_name() << " | "
            << setw(17) << items[i]->get_category() << " | "
            << setw(14) << items[i]->get_price() << " | " << endl
            << setw(71) << setfill('-') << "" << setfill(' ') << endl;
    }

    os << "Итого: " << calculate_total() << endl;

    os.unsetf(ios::fixed);
    os.unsetf(ios::showpoint);
```

```
    os.unsetf(ios::right);
}

bool Order::save_to_file(const string& filename) {
    ofstream out(filename);
    if (!out) return false;
    print_receipt(out);
    out.close();
    return true;
}

void Order::clear() {
    item_count = 0;
    total_price = 0;
}

Clothing::Clothing()
    : name(""), brand(""), size(""), color(""), price(0), quantity(0) {
}

Clothing::Clothing(string name, string brand, string size, string color,
    double price, int quantity)
    : name(name), brand(brand), size(size), color(color),
    price(price), quantity(quantity) {
}

void Clothing::print_info() {
    cout << "Название: " << name
        << ", Бренд: " << brand
        << ", Размер: " << size
        << ", Цвет: " << color
        << ", Цена: " << price
        << ", Количество: " << quantity;
}

Clothing::~Clothing() {}

Jacket::Jacket() : Clothing(), season("") {}

Jacket::Jacket(string name, string brand, string size, string color,
    double price, int quantity, string season)
    : Clothing(name, brand, size, color, price, quantity), season(season) {
}

void Jacket::print_info() { Clothing::print_info(); cout << ", Сезон: " << season; }
string Jacket::get_category() { return "Куртка"; }
Jacket::~Jacket() {}

Trousers::Trousers() : Clothing(), fit("") {}

Trousers::Trousers(string name, string brand, string size, string color,
    double price, int quantity, string fit)
    : Clothing(name, brand, size, color, price, quantity), fit(fit) {
}

void Trousers::print_info() { Clothing::print_info(); cout << ", Посадка: " << fit;
}
string Trousers::get_category() { return "Брюки"; }
Trousers::~Trousers() {}

Shirt::Shirt() : Clothing(), sleeve_type("") {}

Shirt::Shirt(string name, string brand, string size, string color,
    double price, int quantity, string sleeve_type)
    : Clothing(name, brand, size, color, price, quantity),
```

```
    sleeve_type(sleeve_type) {
}

void Shirt::print_info() { Clothing::print_info(); cout << ", Рукав: " <<
sleeve_type; }
string Shirt::get_category() { return "Рубашка"; }
Shirt::~Shirt() {}

Shop::Shop()
    : shop_name(""), catalog(nullptr), orders(nullptr),
    catalog_count(0), catalog_capacity(0),
    orders_count(0), orders_capacity(0) {
}

Shop::Shop(string shop_name)
    : shop_name(shop_name), catalog(nullptr), orders(nullptr),
    catalog_count(0), catalog_capacity(0),
    orders_count(0), orders_capacity(0) {
}

Shop::~Shop() {
    for (int i = 0; i < catalog_count; ++i) delete catalog[i];
    delete[] catalog;
    for (int i = 0; i < orders_count; ++i) delete orders[i];
    delete[] orders;
}

void Shop::add_to_catalog(Clothing* item) {
    if (catalog_count >= catalog_capacity) {
        int new_cap = (catalog_capacity == 0) ? 4 : catalog_capacity * 2;
        Clothing** new_catalog = new Clothing * [new_cap];
        for (int i = 0; i < catalog_count; ++i) new_catalog[i] = catalog[i];
        delete[] catalog;
        catalog = new_catalog;
        catalog_capacity = new_cap;
    }
    catalog[catalog_count++] = item;
}

void Shop::remove_from_catalog(int index) {
    if (index < 0 || index >= catalog_count) return;
    delete catalog[index];
    for (int i = index; i < catalog_count - 1; ++i) catalog[i] = catalog[i + 1];
    --catalog_count;
}

void Shop::edit_catalog_item(int index, Clothing* new_item) {
    if (index < 0 || index >= catalog_count || !new_item) return;
    delete catalog[index];
    catalog[index] = new_item;
}

void Shop::add_order(const Customer& customer) {
    if (orders_count >= orders_capacity) {
        int new_cap = (orders_capacity == 0) ? 4 : orders_capacity * 2;
        Order** new_orders = new Order * [new_cap];
        for (int i = 0; i < orders_count; ++i) new_orders[i] = orders[i];
        delete[] orders;
        orders = new_orders;
        orders_capacity = new_cap;
    }
    string date;
    getline(cin, date);
    orders[orders_count] = new Order(customer, date, orders_count + 1);
    ++orders_count;
```

```
}

void Shop::find_item(const string& name) {
    bool found = false;
    for (int i = 0; i < catalog_count; ++i) {
        if (catalog[i]->get_name() == name) {
            catalog[i]->print_info();
            cout << endl;
            found = true;
        }
    }
    if (!found) cout << "Товар не найден.\n";
}

void Shop::show_catalog(ostream& os) {
    os.setf(ios::fixed, ios::floatfield);
    os << setprecision(2);
    os.setf(ios::showpoint);
    os.setf(ios::right, ios::adjustfield);

    os << setw(120) << setfill('-') << "" << setfill(' ') << endl
        << setw(8) << "№ | "
        << setw(23) << "Название | "
        << setw(18) << "Бренд | "
        << setw(13) << "Размер | "
        << setw(13) << "Цвет | "
        << setw(15) << "Цена | "
        << setw(14) << "Кол-во | "
        << setw(16) << "Категория |" << endl
        << setw(120) << setfill('-') << "" << setfill(' ') << endl;

    for (int i = 0; i < catalog_count; ++i) {
        os << setw(5) << i + 1 << " | "
            << setw(20) << catalog[i]->get_name() << " | "
            << setw(15) << catalog[i]->get_brand() << " | "
            << setw(10) << catalog[i]->get_size() << " | "
            << setw(10) << catalog[i]->get_color() << " | "
            << setw(12) << catalog[i]->get_price() << " | "
            << setw(10) << catalog[i]->get_quantity() << " | "
            << setw(15) << catalog[i]->get_category() << " |" << endl
            << setw(120) << setfill('-') << "" << setfill(' ') << endl;
    }

    os.unsetf(ios::fixed);
    os.unsetf(ios::showpoint);
    os.unsetf(ios::right);
}

void Shop::show_orders(ostream& os) {
    if (orders_count == 0) { os << "Заказов нет.\n"; return; }
    for (int i = 0; i < orders_count; ++i) {
        orders[i]->print_receipt(os);
        os << "--------------------\n";
    }
}

bool Shop::save_all_to_file(const string& filename) {
    ofstream out(filename);
    if (!out) return false;
    out << "Магазин: " << shop_name << "\n\n";
    out << "Каталог:\n";
    show_catalog(out);
    out << "\nЗаказы:\n";
    show_orders(out);
    out.close();
```

```
    ifstream in(filename);
    if (!in) return false;
    string line;
    while (getline(in, line)) {}
    if (in.bad())           cout << "Проверка потока: bad.\n";
    else if (in.eof())      cout << "Проверка потока: eof.\n";
    else if (in.fail())     cout << "Проверка потока: fail.\n";
    else if (in.good())     cout << "Проверка потока: good.\n";
    in.clear();
    return true;
}

//lab1_main.cpp
#include <iostream>
#include <fstream>
#include <iomanip>
#include <windows.h>
#include "l1_class_header.h"
#define FILE_NAME "lab1_file.txt"

using namespace std;

Clothing* create_clothing_from_input() {
    int type;
    cout << "Тип: 1-Куртка 2-Брюки 3-Рубашка: ";
    cin >> type;
    cin.ignore(10000, '\n');

    string name, brand, size, color, extra;
    double price;
    int quantity;

    cout << "Название: ";
    getline(cin, name);
    cout << "Бренд: ";
    getline(cin, brand);
    cout << "Размер: ";
    getline(cin, size);
    cout << "Цвет: ";
    getline(cin, color);
    cout << "Цена: ";
    cin >> price;
    cout << "Количество: ";
    cin >> quantity;
    cin.ignore(10000, '\n');

    if (type == 1) {
        cout << "Сезон: "; getline(cin, extra);
        return new Jacket(name, brand, size, color, price, quantity, extra);
    }
    else if (type == 2) {
        cout << "Посадка: "; getline(cin, extra);
        return new Trousers(name, brand, size, color, price, quantity, extra);
    }
    else {
        cout << "Тип рукава: "; getline(cin, extra);
        return new Shirt(name, brand, size, color, price, quantity, extra);
    }
}

void edit_clothing_fields(Clothing* item) {
    cout << "Что редактировать?\n";
    cout << "1. Название\n2. Бренд\n3. Размер\n4. Цвет\n5. Цену\n6. Количество\n";
    cout << "0. Отмена\n";
```

```
    cout << "Выбор: ";
    int c; cin >> c; cin.ignore(10000, '\n');

    string s; double p; int q;
    switch (c) {
    case 1: cout << "Новое название: ";
        getline(cin, s); item->set_name(s); break;
    case 2: cout << "Новый бренд: ";
        getline(cin, s); item->set_brand(s); break;
    case 3: cout << "Новый размер: ";
        getline(cin, s); item->set_size(s); break;
    case 4: cout << "Новый цвет: ";
        getline(cin, s); item->set_color(s); break;
    case 5: cout << "Новая цена: ";
        cin >> p; cin.ignore(10000, '\n'); item->set_price(p); break;
    case 6: cout << "Новое кол-во: ";
        cin >> q; cin.ignore(10000, '\n'); item->set_quantity(q); break;
    default: cout << "Отмена.\n";
        break;
    }
}

int main() {
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    Shop shop("Одежда/by");

    shop.add_to_catalog(new Jacket("Зимняя куртка", "Nike", "L", "Черный", 12000, 5,
"Зима"));
    shop.add_to_catalog(new Trousers("Джинсы", "Levi's", "32", "Синий", 5000, 10,
"Regular"));
    shop.add_to_catalog(new Shirt("Рубашка", "H&M", "M", "Белый", 2500, 20,
"Длинный"));

    int choice = 0;

    do {
        system("cls");
        cout << "==========Меню магазина одежды==========\n";
        cout << "1. Сделать заказ\n";
        cout << "2. Просмотреть каталог\n";
        cout << "3. Просмотреть информацию о заказах\n";
        cout << "4. Изменить название магазина\n";
        cout << "5. Управление каталогом\n";
        cout << "6. Управление заказом\n";
        cout << "0. Сохранить и выйти\n";
        cout << "========================================\n\n";
        cout << "Ваш выбор: ";

        cin >> choice;
        if (cin.fail()) { cin.clear(); cin.ignore(10000, '\n'); choice = -1; }

        switch (choice) {
        case 1: {
            system("cls");
            cin.ignore(10000, '\n');

            string name, phone, address;
            int card_id;

            cout << "Введите имя: ";
            getline(cin, name);
            cout << "Введите телефон: ";
            getline(cin, phone);
```

```
            cout << "Введите адрес: ";
            getline(cin, address);
            cout << "Введите номер карты: ";
            cin >> card_id;

            Customer customer(name, phone, address, card_id);
            shop.add_order(customer);

            int order_idx = shop.get_orders_count() - 1;
            Order& order = shop.get_order(order_idx);

            char more = 'y';
            while (more == 'y' || more == 'Y') {
                shop.show_catalog(cout);
                cout << "Выберите номер товара (Выход - 0): ";
                int item_idx; cin >> item_idx;

                if (item_idx > 0 && item_idx <= shop.get_catalog_count()) {
                    order.add_item(shop.get_catalog_item(item_idx - 1));
                    shop.get_catalog_item(item_idx - 1)-
>set_quantity(shop.get_catalog_item(item_idx - 1)->get_quantity() - 1);
                    cout << "Товар добавлен.\n";
                }

                else if (item_idx == 0) {
                    break;
                }

                else if (shop.get_catalog_item(item_idx - 1)->get_quantity() == 0) {
                    cout << "Товара нет на складе.\n";
                }

                else {
                    cout << "Неверный номер.\n";
                }

                cout << "Добавить ещё? (y/n): ";
                cin >> more;
            }

            order.calculate_total();
            if (order.calculate_total() > 0) {
                cout.setf(ios::fixed, ios::floatfield);
                cout << setprecision(2) << "Заказ оформлен. Итоговая сумма: "
                    << order.calculate_total() << endl;
                cout.unsetf(ios::fixed);
            }
            else cout << "Заказ пуст. Отмена";

            system("pause");
            break;
        }

        case 2: {
            system("cls");
            shop.show_catalog(cout);
            system("pause");
            break;
        }

        case 3: {
            system("cls");
            shop.show_orders(cout);
            system("pause");
            break;
```

```
        }

        case 4: {
            system("cls");
            cin.ignore(10000, '\n');
            string shopName;
            cout << "Введите название магазина: ";
            getline(cin, shopName);
            shop.set_shop_name(shopName);
            cout << "Название магазина обновлено.\n";
            system("pause");
            break;
        }

        case 5: {
            int sub = 0;
            do {
                system("cls");
                cout << "---Управление каталогом---\n";
                cout << "1. Добавить товар\n";
                cout << "2. Редактировать товар\n";
                cout << "3. Удалить товар\n";
                cout << "0. Назад\n";
                cout << "Выбор: ";
                cin >> sub;
                if (cin.fail()) { cin.clear(); cin.ignore(10000, '\n'); sub = -1; }

                if (sub == 1) {
                    system("cls");
                    cin.ignore(10000, '\n');
                    Clothing* item = create_clothing_from_input();
                    shop.add_to_catalog(item);
                    cout << "Товар добавлен в каталог.\n";
                    system("pause");
                }
                else if (sub == 2) {
                    system("cls");
                    shop.show_catalog(cout);
                    if (shop.get_catalog_count() == 0) { system("pause"); continue;
}
                    cout << "Введите номер товара для редактирования: ";
                    int idx; cin >> idx;
                    cin.ignore(10000, '\n');
                    if (idx > 0 && idx <= shop.get_catalog_count()) {
                        edit_clothing_fields(shop.get_catalog_item(idx - 1));
                        cout << "Товар изменён.\n";
                    }
                    else {
                        cout << "Неверный номер.\n";
                    }
                    system("pause");
                }
                else if (sub == 3) {
                    system("cls");
                    shop.show_catalog(cout);
                    if (shop.get_catalog_count() == 0) { system("pause"); continue;
}
                    cout << "Введите номер товара для удаления: ";
                    int idx; cin >> idx;
                    if (idx > 0 && idx <= shop.get_catalog_count()) {
                        shop.remove_from_catalog(idx - 1);
                        cout << "Товар удалён.\n";
                    }
                    else {
                        cout << "Неверный номер.\n";
```

```
                }
                system("pause");
            }
        } while (sub != 0);
        break;
    }

    case 6: {
        system("cls");
        if (shop.get_orders_count() == 0) {
            cout << "Заказов нет.\n";
            system("pause");
            break;
        }
        shop.show_orders(cout);
        cout << "Введите номер заказа: ";
        int order_idx; cin >> order_idx;
        if (order_idx < 1 || order_idx > shop.get_orders_count()) {
            cout << "Неверный номер.\n";
            system("pause");
            break;
        }
        Order& order = shop.get_order(order_idx - 1);

        int sub = 0;
        do {
            system("cls");
            cout << "---Управление заказом №" << order_idx << "---\n";
            order.print_receipt(cout);
            cout << "\n1. Добавить товар в заказ\n";
            cout << "2. Редактировать товар в заказе\n";
            cout << "3. Удалить товар из заказа\n";
            cout << "0. Назад\n";
            cout << "Выбор: ";
            cin >> sub;
            if (cin.fail()) { cin.clear(); cin.ignore(10000, '\n'); sub = -1; }

            if (sub == 1) {
                system("cls");
                shop.show_catalog(cout);
                if (shop.get_catalog_count() == 0) { system("pause"); continue;
}
                cout << "Введите номер товара: ";
                int idx; cin >> idx;
                if (idx > 0 && idx <= shop.get_catalog_count()) {
                    order.add_item(shop.get_catalog_item(idx - 1));
                    order.calculate_total();
                    cout << "Товар добавлен в заказ.\n";
                }
                else {
                    cout << "Неверный номер.\n";
                }
                system("pause");
            }
            else if (sub == 2) {
                system("cls");
                order.print_receipt(cout);
                if (order.get_item_count() == 0) { system("pause"); continue; }
                cout << "Введите номер позиции заказа для редактирования: ";
                int idx; cin >> idx;
                if (idx > 0 && idx <= order.get_item_count()) {
                    shop.show_catalog(cout);
                    cout << "Введите номер нового товара из каталога: ";
                    int new_idx; cin >> new_idx;
                    if (new_idx > 0 && new_idx <= shop.get_catalog_count()) {
```

```
                            order.edit_item(idx - 1, shop.get_catalog_item(new_idx -
1));
                            order.calculate_total();
                            cout << "Позиция заказа обновлена.\n";
                        }
                        else {
                            cout << "Неверный номер товара.\n";
                        }
                    }
                    else {
                        cout << "Неверная позиция.\n";
                    }
                    system("pause");
                }
                else if (sub == 3) {
                    system("cls");
                    order.print_receipt(cout);
                    if (order.get_item_count() == 0) { system("pause"); continue; }
                    cout << "Введите номер позиции для удаления: ";
                    int idx; cin >> idx;
                    if (idx > 0 && idx <= order.get_item_count()) {
                        order.remove_item(idx - 1);
                        order.calculate_total();
                        cout << "Позиция удалена.\n";
                    }
                    else {
                        cout << "Неверный номер.\n";
                    }
                    system("pause");
                }
            } while (sub != 0);
            break;
        }

        case 0: {
            system("cls");
            if (shop.save_all_to_file(FILE_NAME))
                cout << "Данные сохранены в " << FILE_NAME << endl;
            else
                cout << "Ошибка сохранения.\n";
            break;
        }

        default:
            cout << "Неверный выбор" << endl;
            system("pause");
            break;
        }
    } while (choice != 0);

    cout << "До свидания" << endl;
    return 0;
}
```

Примеры работы программы приведены на русунках 1, 2, 3, 4. На рисунке 5 изображён пример вывода данных в текстовый файл. UML диаграмма созданных классов показана на рисунке 6.

```
==========Меню магазина одежды==========
1. Сделать заказ
2. Просмотреть каталог
3. Просмотреть информацию о заказах
4. Изменить название магазина
5. Управление каталогом
6. Управление заказом
0. Сохранить и выйти
========================================

Ваш выбор: 1
Введите имя: Артём
Введите телефон: +37533257633
Введите адрес: ул. Калиновского д.32
Введите номер карты: 432653
```

№	Название	Бренд	Размер	Цвет	Цена	Кол-во	Категория
1	Зимняя куртка	Nike	L	Черный	12000.00	5	Куртка
2	Джинсы	Levi's	32	Синий	5000.00	10	Брюки
3	Рубашка	H&M	M	Белый	2500.00	20	Рубашка

```
Выберите номер товара (Выход - 0): 
```

Рисунок 1 – Пример работы программы

```
Выберите номер товара (Выход - 0): 1
Товар добавлен.
Добавить ещё? (y/n): y
```

№	Название	Бренд	Размер	Цвет	Цена	Кол-во	Категория
1	Зимняя куртка	Nike	L	Черный	12000.00	4	Куртка
2	Джинсы	Levi's	32	Синий	5000.00	10	Брюки
3	Рубашка	H&M	M	Белый	2500.00	20	Рубашка

```
Выберите номер товара (Выход - 0): 2
Товар добавлен.
Добавить ещё? (y/n): y
```

№	Название	Бренд	Размер	Цвет	Цена	Кол-во	Категория
1	Зимняя куртка	Nike	L	Черный	12000.00	4	Куртка
2	Джинсы	Levi's	32	Синий	5000.00	9	Брюки
3	Рубашка	H&M	M	Белый	2500.00	20	Рубашка

```
Выберите номер товара (Выход - 0): 0
Заказ оформлен. Итоговая сумма: 17000.00
Press any key to continue . . .
==========Меню магазина одежды==========
1. Сделать заказ
2. Просмотреть каталог
3. Просмотреть информацию о заказах
4. Изменить название магазина
5. Управление каталогом
6. Управление заказом
0. Сохранить и выйти
========================================

Ваш выбор: 2
```

Рисунок 2 – Пример работы программы

```
==========Меню магазина одежды==========
1. Сделать заказ
2. Просмотреть каталог
3. Просмотреть информацию о заказах
4. Изменить название магазина
5. Управление каталогом
6. Управление заказом
0. Сохранить и выйти
========================================

Ваш выбор: 3
Заказ №1 от
Имя: Артём, Телефон: +37533257633, Адрес: ул. Калиновского д.32, Карта: 432653
Товары:
------------------------------------------------------------------------
     № |              Название |         Категория |           Цена |
------------------------------------------------------------------------
     1 |         Зимняя куртка |           Куртка |       12000.00 |
------------------------------------------------------------------------
     2 |                Джинсы |            Брюки |        5000.00 |
------------------------------------------------------------------------
Итого: 17000.00
------------------
Press any key to continue . . .
```

Рисунок 3 – Пример работы программы

```
Press any key to continue . . .
==========Меню магазина одежды==========
1. Сделать заказ
2. Просмотреть каталог
3. Просмотреть информацию о заказах
4. Изменить название магазина
5. Управление каталогом
6. Управление заказом
0. Сохранить и выйти
========================================

Ваш выбор: 5
---Управление каталогом---
1. Добавить товар
2. Редактировать товар
3. Удалить товар
0. Назад
Выбор: 1
Тип: 1-Куртка 2-Брюки 3-Рубашка: 1
Название: Куртка
Бренд: Supreme
Размер: M
Цвет: красный
Цена: 329
Количество: 8
Сезон: зима
Товар добавлен в каталог.
Press any key to continue . . .
```

Рисунок 4 – Пример работы программы

```
Магазин: Одежда/by

Каталог:
-------------------------------------------------------------------------------------------------------------
    № |            Название |          Бренд |     Размер |       Цвет |        Цена |      Кол-во |      Категория |
-------------------------------------------------------------------------------------------------------------
    1 |       Зимняя куртка |           Nike |          L |     Черный |    12000.00 |           4 |          Куртка |
-------------------------------------------------------------------------------------------------------------
    2 |             Джинсы |         Levi's |         32 |      Синий |     5000.00 |           9 |          Брюки |
-------------------------------------------------------------------------------------------------------------
    3 |            Рубашка |            H&M |          M |      Белый |     2500.00 |          20 |         Рубашка |
-------------------------------------------------------------------------------------------------------------
    4 |             Куртка |        Supreme |          M |    красный |      329.00 |           8 |          Куртка |
-------------------------------------------------------------------------------------------------------------

Заказы:
Заказ №1 от
Имя: Артём, Телефон: +37533257633, Адрес: ул. Калиновского д.32, Карта: 432653
Товары:
-----------------------------------------------------------------------
      № |             Название |       Категория |          Цена |
-----------------------------------------------------------------------
      1 |        Зимняя куртка |          Куртка |      12000.00 |
-----------------------------------------------------------------------
      2 |               Джинсы |           Брюки |       5000.00 |
-----------------------------------------------------------------------
Итого: 17000.00
-------------------
```

Рисунок 5 – Пример вывода в файл

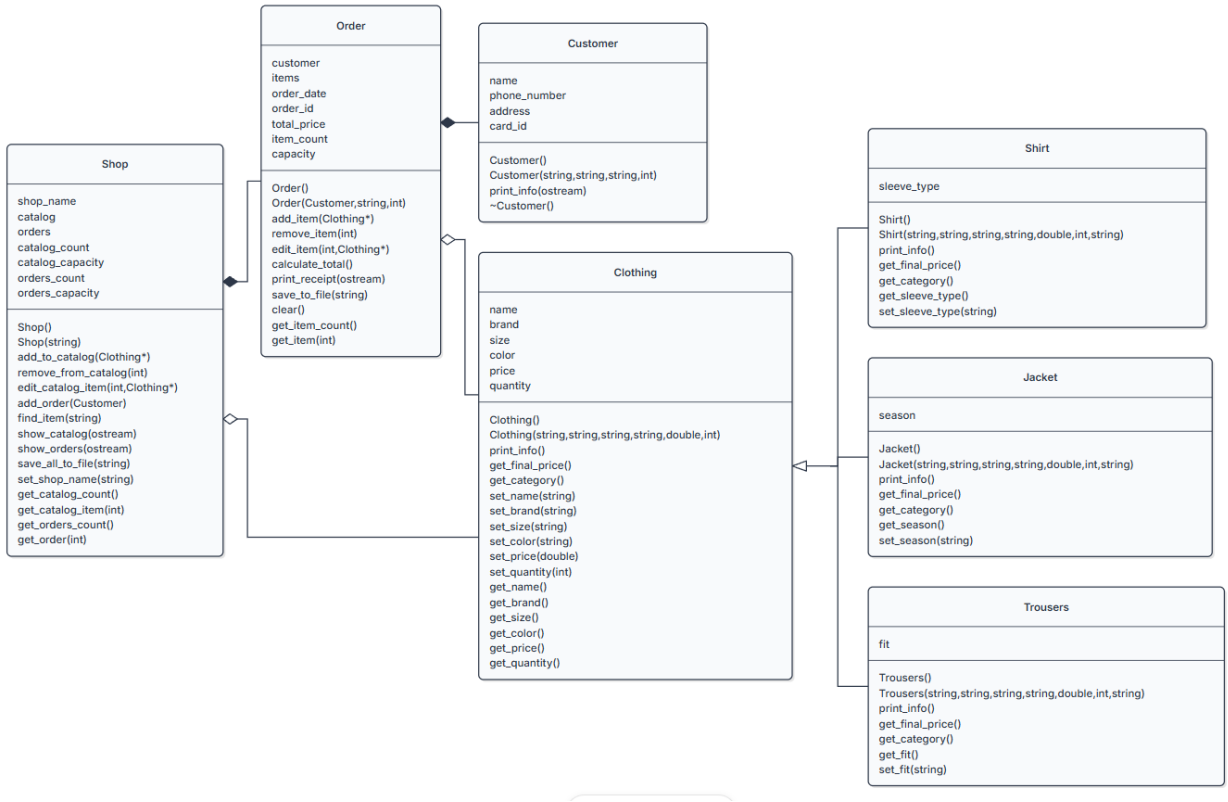


Рисунок 6 – UML диаграмма созданных классов

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое поток в языке C++?

Поток в C++ — это объект потокового класса, через который выполняется ввод или вывод данных (последовательность байтов, связанная с источником или приёмником).

2. Какой класс является базовым для большинства потоковых классов ввода-вывода?

Базовый класс — ios_base (от него происходят basic_ios, а далее istream/ostream/iostream).

3. Истинно ли утверждение «потоки делятся на входные и выходные»?

Истинно. Потоки делятся на входные (istream, ifstream), выходные (ostream, ofstream) и двунаправленные (iostream, fstream).

4. Что определяет флаг skipws при его использовании с объектом cin?

skipws управляет пропуском пробельных символов (пробелов, табов, переводов строк) при извлечении из cin. Установлен по умолчанию; noskipws отключает пропуск.

5. Напишите сигнатуру функции main(), позволяющую программе использовать аргументы командной строки. Запустите приложение, передав аргументы в функцию main(). В каких ситуация необходимо передавать аргументы в функцию main()?

int main(int argc, char* argv[]), где argc — количество аргументов, argv — массив строк-аргументов. Аргументы нужны, когда программе передаются параметры из командной строки (имя файла, режим работы, ключи и т.п.).

6. Что собой представляют стандартные потоки ввода/вывода?

Стандартные потоки ввода/вывода — предопределённые объекты: cin (клавиатура), cout (экран), cerr (небуферизованный поток ошибок), clog (буферизованный поток ошибок).

7. Какой заголовочный файл необходимо подключить для работы с потоками ввода/вывода?

Заголовочный файл — <iostream> (для файловых потоков — <fstream>, для манипуляторов — <iomanip>).

8. Что представляет собой поток форматированного вывода? Какие функции обеспечивают форматированный вывод?

Поток форматированного вывода — это вывод, при котором данные преобразуются по заданным правилам (ширина поля, точность, система счисления, выравнивание и т.д.). Обеспечивают функции: setf(), unsetf(), width(), precision(), fill(), а также манипуляторы setw(), setprecision(), setfill(), setiosflags(), resetiosflags().

9. Для чего предназначена библиотека ввода/вывода iomanip?

<iomanip> предназначена для использования манипуляторов форматирования ввода-вывода с аргументами: setw, setprecision, setfill, setiosflags, resetiosflags.

10. Напишите оператор, который позволит считать с консоли строку данных?

getline(cin, str); // для std::string

cin.getline(buf, size); // для char-массива

Выводы. В ходе выполнения лабораторной работы были изучены особенности работы с потоками ввода/вывода в C++. Были рассмотрены различные флаги и то, для чего они применяются. Также были изучены новые манипуляторы потока и разобран принцип их работы.